

Maklayn

Relatório técnico do projeto

Funcionamento, arquitetura, tecnologias e composição

Criador: Jhon Maklayn Mozer dos Santos

Data de início informada: 12/09/2026

Versão analisada: checkpoint 567c8ac

Relatório preparado: 07/10/2026

Documento técnico de referência do MVP Maklayn

Sumário

1	Resumo executivo	1
1.1	Leitura correta das métricas	1
2	Para que a Maklayn foi feita	1
2.1	Público e problemas atendidos	1
2.2	Princípios de resposta	2
3	Como o sistema funciona	2
3.1	Visão geral do fluxo	2
3.2	Chat inteligente	2
3.3	Copiloto de código	3
3.4	Arquivos grandes e ZIPs	3
3.5	Documentos, PDF e ZIP de projeto	3
4	Arquitetura técnica	3
4.1	Frontend	3
4.2	Backend	4
4.3	Dados e autenticação	4
4.4	Build e execução	4
5	Tecnologias e tipos de código usados	4
5.1	Linguagens e formatos	4
5.2	Principais tecnologias	5
6	Linhas por pasta	5
6.1	Principais arquivos da aplicação	6
7	Como o projeto foi construído	6
7.1	Etapa 1 — Base e decisão de produto	6
7.2	Etapa 2 — Identidade e interface	6
7.3	Etapa 3 — Inteligência e revisão	6
7.4	Etapa 4 — Arquivos e projetos	7
7.5	Etapa 5 — Qualidade e checkpoint	7
8	O que está pronto e o que falta	7
8.1	Entregue no MVP	7
8.2	Próxima fase recomendada	8
9	Conclusão	8
10	Anexo A — Contratos de entrada e saída	9
10.1	Chat	9
10.2	Assistência de código	9
11	Anexo B — Ciclo de uma requisição	10
11.1	Estados de interface	10
12	Anexo C — Como o copiloto de código raciocina	11
12.1	Agente implementador	11
12.2	Agente revisor	11
12.3	Testes antes da entrega	11

12.4	Exemplo de checklist técnico	11
13	Anexo D — Análise incremental de arquivos	12
13.1	Pseudofluxo	12
13.2	Binários	12
13.3	Segurança do upload	12
14	Anexo E — Empacotamento e exportação	13
14.1	Estrutura do ZIP	13
14.2	Nomes e colisões	13
14.3	Evolução para storage	13
15	Anexo F — Modelo de dados e persistência	14
15.1	Migrações	14
16	Anexo G — Modelo de segurança	15
16.1	Riscos principais	15
16.2	Privacidade	15
17	Anexo H — Estratégia de testes	16
17.1	Camadas	16
17.2	Casos de borda	16
17.3	Critério de entrega de código	16
18	Anexo I — Build, deploy e ambientes	17
18.1	Ambientes	17
18.2	Health check	17
18.3	CI sugerida	17
19	Anexo J — Observabilidade e operação	18
19.1	Logs estruturados	18
19.2	Estados de job	18
19.3	Incidentes	18
20	Anexo K — Mapa de módulos da interface	19
20.1	Acessibilidade	19
21	Anexo L — Glossário técnico	20
22	Encerramento técnico ampliado	20

1 Resumo executivo

A Maklayn é uma plataforma web de inteligência artificial voltada para estudo, pesquisa, programação e organização de projetos. O produto foi concebido como uma bancada digital: em vez de oferecer apenas uma caixa de conversa, reúne módulos especializados para conversar, pesquisar, comparar respostas, escrever código, estudar para o ENEM e organizar projetos.

O MVP analisado foi construído sobre uma aplicação web full-stack em TypeScript, com frontend React servido por Vite, backend Express, procedimentos tRPC, integração com o LLM gerenciado da plataforma Manus e uma camada Drizzle preparada para banco de dados. A interface prioriza uma experiência editorial e operacional: barra lateral contextual, canvas amplo, editor de código, estados de carregamento, feedbacks visuais e modo claro/escuro.

A versão atual inclui, além do chat, um copiloto de código com dois agentes simultâneos, análise incremental de arquivos grandes, leitura de entradas de ZIP, anexos universais, exportação de PDF e criação de ZIP de projeto. A execução real de código em sandbox, a persistência privada de documentos e a pesquisa web ao vivo continuam explicitamente identificadas como próximas fases.

1.1 Leitura correta das métricas

As métricas deste relatório foram calculadas sobre os arquivos versionados pelo Git, excluindo `node_modules`, `dist`, `cache`s e artefatos locais. O número total inclui arquivos de configuração e o grande `pnpm-lock.yaml`; por isso, o relatório também apresenta o número mais útil para medir a aplicação: código executável e estilos da solução.

Métrica	Valor
Arquivos versionados	135
Linhas versionadas totais	23.792
Arquivos de aplicação (TS, TSX, CSS, JS, HTML, SQL)	112
Linhas de aplicação	13.931
Linhas do <code>pnpm-lock.yaml</code>	9.172
Último checkpoint analisado	567c8ac

2 Para que a Maklayn foi feita

A finalidade do projeto é oferecer um ambiente único para transformar perguntas e ideias em trabalho verificável. O nome Maklayn representa uma ferramenta de construção intelectual: o usuário fornece contexto, documentos, código ou um objetivo; a plataforma organiza a tarefa e retorna uma resposta com explicação, revisão e próximos passos.

2.1 Público e problemas atendidos

- **Estudantes:** organização de estudos, trilhas do ENEM, questões, feedback pedagógico e revisão de redação.
- **Pesquisadores:** síntese de hipóteses, organização de fontes e separação entre fato, inferência e incerteza.
- **Desenvolvedores:** geração, revisão, explicação, testes e auditoria defensiva de código.

- **Criadores de projetos:** transformação de um prompt em arquitetura, árvore de arquivos, implementação inicial, testes e instruções reproduzíveis.
- **Usuários que trabalham com documentos:** anexação de formatos variados, análise por partes e empacotamento em ZIP.

2.2 Princípios de resposta

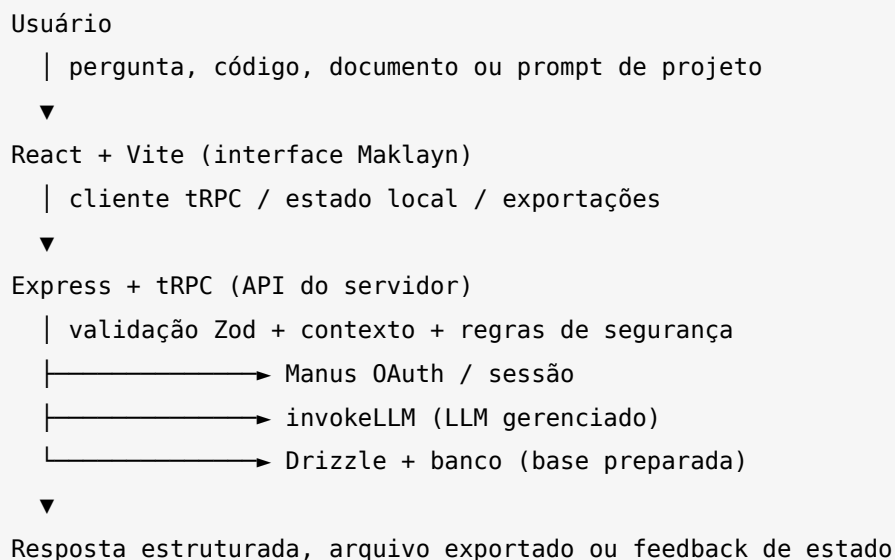
A Maklayn foi configurada para responder diretamente a pedidos permitidos, sem apresentar como verificado aquilo que não foi verificado. O comportamento esperado é:

1. entender a tarefa e o contexto fornecido;
2. separar fatos, inferências, opiniões e incertezas;
3. não inventar fontes, URLs, autores ou resultados;
4. adaptar profundidade e formato ao pedido;
5. quando houver risco técnico, apontar limitações e oferecer uma alternativa defensiva;
6. quando uma etapa depender de execução, compilação, rede ou credencial, declarar essa dependência.

3 Como o sistema funciona

3.1 Visão geral do fluxo

O fluxo principal da Maklayn é uma cadeia cliente-servidor. O navegador apresenta a interface, mantém o estado imediato da sessão e envia pedidos pelo cliente tRPC. O servidor recebe o pedido por uma rota tRPC, valida os dados com Zod, monta o contexto de sistema e chama o cliente LLM no backend. A resposta retorna ao navegador sem expor credenciais do provedor.



3.2 Chat inteligente

O procedimento `ai.chat` recebe uma lista de mensagens com papéis `user`, `assistant` ou `system`. O servidor acrescenta um prompt de sistema da Maklayn, encaminha o contexto ao LLM gerenciado e devolve `content`. Quando o serviço não responde, o frontend usa um modo demonstrativo explícito, sem fabricar citações ou fingir que uma resposta foi verificada.

3.3 Copiloto de código

O procedimento `ai.codeAssist` recebe operação, linguagem, tarefa, código e nome de arquivo. As operações disponíveis são:

- gerar código;
- revisar código;
- criar testes;
- explicar código;
- auditar segurança;
- montar projeto.

Para cada pedido, dois agentes são acionados em paralelo. O primeiro atua como implementador ou analista principal. O segundo faz uma revisão independente, procurando bugs, regressões, casos de borda, requisitos ausentes e testes que ainda precisam passar. A resposta final identifica as duas partes para que o usuário distinga implementação de revisão.

A operação de montagem de projeto pede uma arquitetura completa, árvore de arquivos, implementação inicial por arquivo, testes, variáveis de ambiente, comandos de instalação e checklist de produção. A Maklayn não afirma que criou ou executou os arquivos quando apenas gerou o conteúdo.

3.4 Arquivos grandes e ZIPs

Arquivos de texto e código são divididos em partes de aproximadamente 12.000 caracteres. Cada parte é enviada ao procedimento `ai.analyzeFileChunk`, que mantém um resumo acumulado. Dois agentes analisam cada parte: um leitor preserva estrutura, nomes e decisões; um auditor acrescenta riscos, bugs, segredos aparentes e perguntas abertas.

Quando o arquivo é ZIP, o frontend expande suas entradas com `fflate`, ignora diretórios vazios e identifica extensões binárias que não devem ser tratadas como texto. Entradas legíveis seguem o mesmo fluxo incremental. Entradas binárias são preservadas para exportação, ainda que a interpretação profunda exija um parser específico.

3.5 Documentos, PDF e ZIP de projeto

O CodeLab possui um campo universal de arquivos, sem filtro de extensão, e permite múltiplos anexos. O estado dos anexos é mantido na sessão do navegador. O botão PDF usa `jsPDF` para gerar um documento com o conteúdo atual ou o relatório da Maklayn. O botão ZIP usa `fflate` para criar `maklayn-project.zip` com:

- `maklayn-project/src/current-code.txt`;
- `maklayn-project/README.md`;
- `maklayn-project/maklayn-analysis.md`;
- `maklayn-project/attachments/`, contendo os anexos originais.

4 Arquitetura técnica

4.1 Frontend

O frontend está em `client/` e foi desenvolvido em React com TypeScript e TSX. `client/src/App.tsx` concentra o shell da aplicação, a navegação entre módulos, o estado local do MVP, o fluxo de autenticação e

a ligação das mutations tRPC. `client/src/components/CodeLab.tsx` implementa o editor, anexos, análise incremental, exportação de PDF e exportação de ZIP. `client/src/index.css` contém o sistema visual, responsividade, temas e estados de interação.

A interface usa componentes do ecossistema Radix UI, ícones lucide-react, sonner para feedbacks e streamdown para renderização de respostas Markdown. O tema claro usa superfícies marfim e destaques creme; o tema escuro usa destaques cinza.

4.2 Backend

O backend está em `server/` e é servido por Express. `server/_core/index.ts` inicia o servidor, integra Vite no desenvolvimento e expõe o health check. `server/_core/trpc.ts` configura o contexto e os procedimentos. `server/routers.ts` define autenticação, chat, copiloto de código e análise incremental de arquivos. `server/_core/llm.ts` encapsula a chamada ao LLM gerenciado.

A regra mais importante da separação é que credenciais e chamadas privilegiadas permanecem no servidor. O navegador recebe apenas o resultado necessário para apresentar a experiência.

4.3 Dados e autenticação

A pasta `drizzle/` contém schema, relações, configuração de migração e a migração inicial de usuários do starter. O fluxo de autenticação preservado é Manus OAuth, com sessão validada no backend. O MVP usa estado local para parte da experiência, enquanto tabelas completas para projetos, conversas, mensagens, fontes e documentos fazem parte da evolução planejada.

4.4 Build e execução

Os scripts principais são:

Comando	Finalidade
<code>pnpm dev</code>	Inicia o servidor de desenvolvimento com TypeScript em modo watch.
<code>pnpm check</code>	Executa o TypeScript compiler sem emitir arquivos.
<code>pnpm test</code>	Executa os testes Vitest do projeto.
<code>pnpm build</code>	Gera o bundle do frontend com Vite e do servidor com esbuild.
<code>pnpm start</code>	Executa o bundle de produção gerado.
<code>pnpm db:push</code>	Gera e aplica migrações Drizzle.
<code>pnpm db:migrate</code>	Aplica migrações já geradas.

5 Tecnologias e tipos de código usados

5.1 Linguagens e formatos

Extensão	Arquivos	Linhas
<code>.tsx</code>	67	9.358
<code>.ts</code>	41	3.420
<code>.json</code>	8	305
YAML	2	9.181

Extensão	Arquivos	Linhas
JavaScript	1	899
.md	3	152
sem extensão	7	182
.css	1	217
.html	1	24
.sql	1	13
.svg	1	6
.patch	1	28

Os valores de YAML são altos principalmente porque `pnpm-lock.yaml` registra a árvore de dependências e versões resolvidas. Ele é importante para reprodutibilidade, mas não equivale a código escrito especificamente para a lógica da Maklayn.

5.2 Principais tecnologias

- **TypeScript:** linguagem principal do frontend e backend, com tipos compartilhados.
- **TSX/React:** composição da interface, componentes, módulos e estados interativos.
- **Vite:** servidor e bundler do frontend.
- **Express:** servidor HTTP e integração com o runtime WebDev.
- **tRPC:** contrato tipado entre frontend e backend.
- **Zod:** validação dos inputs das procedures.
- **Drizzle ORM:** schema e migrações do banco.
- **Manus OAuth:** autenticação e sessão do starter.
- **LLM gerenciado:** geração de respostas e análise assistida no backend.
- **fflate:** leitura de ZIPs e criação do ZIP de projeto.
- **jsPDF:** geração de PDF no navegador.
- **Vitest:** testes automatizados existentes.
- **esbuild:** bundle do servidor para produção.
- **CSS customizado:** identidade visual, responsividade e temas claro/escuro.

6 Linhas por pasta

A tabela abaixo considera os arquivos versionados pelo Git. A pasta [raiz] contém configurações, documentação, lockfile e arquivos de projeto; por isso, seu total não deve ser lido como código de aplicação.

Pasta	Arquivos	Linhas	O que contém
client/	79	10.748	Interface React, páginas, componentes, estilos e manifesto público.
server/	25	2.837	Express, tRPC, autenticação, LLM, storage, helpers e testes.
drizzle/	6	165	Schema, relações, metadados e migração inicial.

Pasta	Arquivos	Linhas	O que contém
shared/	3	63	Tipos e constantes compartilhados entre cliente e servidor.
patches/	1	28	Patch de dependência.
[raiz]	21	9.951	Configurações, documentação, lock-file, Docker e arquivos de build.

6.1 Principais arquivos da aplicação

Arquivo	Linhas	Responsabilidade
client/src/ pages/ComponentShowcase.tsx	1.437	Showcase de componentes do starter.
client/src/components/ui/sidebar.tsx	733	Componente de navegação lateral reutilizável.
server/_core/llm.ts	453	Cliente do LLM gerenciado.
client/src/App.tsx	336	Shell da aplicação e módulos Maklayn.
client/src/components/CodeLab.tsx	208	Editor, copiloto, anexos, PDF e ZIP.
client/src/index.css	217	Sistema visual e responsividade.
server/routers.ts	aprox. 140	Procedures de chat, código e chunks de arquivo.

7 Como o projeto foi construído

7.1 Etapa 1 — Base e decisão de produto

O projeto partiu do template web com banco e servidor habilitados. A decisão foi manter React + Vite + Express + tRPC + Drizzle, evitando reescrever a infraestrutura do starter. O plano separou o que seria demonstrável no MVP do que exigiria serviços de produção, como RAG, storage privado, execução isolada e busca ao vivo.

7.2 Etapa 2 — Identidade e interface

A interface foi desenhada como uma bancada editorial: navegação lateral fixa, cabeçalho de contexto, módulos com propósito claro, cards assimétricos e áreas de trabalho. A marca começou com um símbolo tipográfico e depois recebeu a imagem medieval enviada pelo criador. A paleta foi ajustada para creme no tema claro e cinza no tema escuro.

7.3 Etapa 3 — Inteligência e revisão

O chat foi conectado ao invokeLLM no servidor com fallback transparente. Depois, o copiloto de código foi ampliado para dois agentes paralelos. O contrato passou a distinguir agente implementador e agente revisor, evitando que uma única resposta fosse apresentada como se tivesse sido testada quando não houve execução real.

7.4 Etapa 4 — Arquivos e projetos

A análise de documentos evoluiu de um campo de código para ingestão incremental. Textos são quebrados em chunks; ZIPs são abertos por entrada; anexos são mantidos na sessão; e o usuário pode exportar um pacote ZIP com código, documentação, relatório e arquivos originais. A geração de PDF foi adicionada no cliente com jsPDF.

7.5 Etapa 5 — Qualidade e checkpoint

Cada incremento foi submetido a `pnpm check`, `pnpm test`, `pnpm build`, health check e inspeção de diff. Os commits registrados mostram a evolução do MVP:

Commit	Data	Entrega
c71b05f	06/10/2026	Scaffold inicial.
0dec425	06/10/2026	MVP educacional Maklayn.
d1109d1	06/10/2026	Copiloto de programação.
ae1df14	06/10/2026	Respostas adaptativas e dois agentes.
5a69f11	06/10/2026	Arquivos grandes, ZIPs e blueprints.
bbae334	06/10/2026	Documentos, PDF e exportação ZIP.
d7ba9ec	07/10/2026	Campo universal de arquivos.
a8b7f71	07/10/2026	Nova logo medieval.
567c8ac	07/10/2026	Paleta creme e cinza.

A data de início 12/09/2026 é a data informada pelo criador neste relatório. O histórico Git disponível no ambiente começa com o scaffold em 06/10/2026; as duas datas representam, respectivamente, o início declarado do projeto e o início do histórico técnico recuperado no repositório.

8 O que está pronto e o que falta

8.1 Entregue no MVP

- dashboard responsivo com módulos de Chat, Pesquisa, Arena, Código, ENEM e Projetos;
- chat com integração LLM e fallback demonstrativo explícito;
- respostas de código geradas, revisadas e acompanhadas de testes sugeridos;
- modo de dois agentes em paralelo;
- análise incremental de textos e códigos grandes;
- leitura de ZIPs por entrada;
- campo universal para múltiplos arquivos;
- escrita de Markdown e código no editor;
- exportação de PDF e ZIP de projeto;
- autenticação Manus OAuth preservada;
- health check, manifesto de rotas, build e testes;
- temas claro e escuro e marca visual atualizada.

8.2 Próxima fase recomendada

1. persistir projetos, conversas, mensagens e anexos em banco e storage privado;
2. adicionar parsers dedicados para PDF, DOCX, XLSX, imagens, áudio e outros formatos;
3. implementar fila de indexação e busca semântica/RAG;
4. conectar executor isolado para compilar e testar código real com limites de CPU, memória, tempo e rede;
5. habilitar pesquisa web autorizada com referências rastreáveis;
6. completar controles de privacidade, exportação e exclusão de dados;
7. criar CI, observabilidade, backup e publicação de produção.

9 Conclusão

A Maklayn foi feita para ser uma estação de trabalho de IA, não apenas um chat. Sua arquitetura separa a interface, o servidor, a validação, o LLM, a autenticação e a futura persistência. O resultado atual é um MVP funcional e extensível: já demonstra os principais fluxos de estudo, pesquisa, programação, documentos e exportação, enquanto mantém declaradas as fronteiras que ainda exigem infraestrutura de produção.

O criador registrado neste documento é **Jhon Maklayn Mozer dos Santos**. A data oficial de início informada é **12/09/2026**.

10 Anexo A — Contratos de entrada e saída

A interface entre navegador e servidor é deliberadamente tipada. O frontend não envia um objeto livre esperando que o backend descubra sua intenção; cada operação recebe campos conhecidos, que podem ser validados antes da chamada ao modelo. Essa decisão reduz falhas silenciosas e facilita a evolução da API.

10.1 Chat

O contrato conceitual do chat pode ser representado assim:

```
ChatInput {  
  messages: Message[]  
}  
Message {  
  role: "user" | "assistant" | "system"  
  content: string  
}  
ChatOutput {  
  content: string  
  demo?: boolean  
}
```

A validação deve rejeitar mensagens sem conteúdo, papéis desconhecidos e estruturas que não correspondam ao formato esperado. O backend é a autoridade para acrescentar as instruções de sistema, definir o modelo e controlar limites de contexto. O cliente não tem permissão para escolher credenciais ou alterar a política de segurança do servidor.

10.2 Assistência de código

```
CodeAssistInput {  
  operation: "generate" | "review" | "test" | "explain" |  
    "security" | "project"  
  language: string  
  brief: string  
  code: string  
  fileName?: string  
}  
CodeAssistOutput {  
  results: AgentResult[]  
  mode: "dual-agent"  
}  
AgentResult {  
  agent: "implementer" | "reviewer"  
  content: string  
}
```

Esse desenho é importante porque separa a intenção do usuário do texto gerado. Uma futura versão pode substituir `string` por estruturas como `files`, `patches`, `commands`, `tests` e `warnings`, sem quebrar a ideia central da operação.

11 Anexo B — Ciclo de uma requisição

Uma requisição do chat ou do CodeLab percorre várias camadas. O primeiro passo é a interação do usuário: o texto é capturado por um formulário React, o estado de carregamento é ativado e o botão é desabilitado para evitar envios duplicados. Em seguida, o cliente tRPC serializa o input e envia uma requisição HTTP para o servidor.

No servidor, o contexto tRPC identifica a sessão disponível e encaminha a chamada para o router. O procedimento valida o input, seleciona a operação e prepara o prompt. Em pedidos de código, o prompt inclui linguagem, arquivo, objetivo, restrições de segurança e instrução para diferenciar código pronto de sugestão.

A chamada ao LLM ocorre no backend, onde as credenciais gerenciadas estão disponíveis. O retorno é normalizado para uma resposta textual. No modo dual-agent, duas promessas são executadas em paralelo com `Promise.all`, reduzindo o tempo total em comparação com duas chamadas sequenciais. Se uma chamada falha, o sistema deve preservar o resultado da outra e registrar a falha como revisão incompleta, em vez de fingir que houve consenso.

No navegador, a mutation encerra o estado de carregamento, adiciona o resultado à interface e mostra um toast de sucesso ou erro. O usuário pode copiar o texto, continuar a conversa, salvar o material ou exportá-lo. Esse ciclo mantém uma fronteira clara: a Maklayn pode gerar e revisar, mas não afirma que compilou ou executou algo quando não houve executor real.

11.1 Estados de interface

Estado	Comportamento	Por que existe
idle	Campos disponíveis e ação pronta.	Experiência imediata.
loading	Indicador de pensamento e ação desabilitada.	Evita duplicação.
success	Resposta renderizada e ações secundárias.	Continuidade do trabalho.
error	Mensagem clara e possibilidade de tentar novamente.	Recuperação sem perder contexto.
demo	Aviso explícito de que o LLM não respondeu.	Honestidade operacional.

12 Anexo C — Como o copiloto de código raciocina

O copiloto foi projetado como uma etapa de engenharia assistida, não como um gerador cego de snippets. Cada solicitação começa com um briefing. O briefing precisa declarar o resultado esperado, o ambiente, a linguagem, os arquivos envolvidos e as restrições. Quanto mais concreto o contexto, maior a chance de a resposta ser funcional.

12.1 Agente implementador

O implementador é instruído a produzir uma solução executável em princípio, com imports, tipos, tratamento de erro e integração coerentes. Ele deve evitar pseudocódigo quando o usuário pediu código, indicar arquivos alterados e incluir comandos de uso. Quando o pedido é um projeto inteiro, o resultado deve começar pela arquitetura e depois descer para os arquivos mais importantes.

12.2 Agente revisor

O revisor recebe o mesmo objetivo e a mesma base. Ele procura variáveis não definidas, imports inexistentes, tipos incompatíveis, caminhos errados, condições de corrida, validação ausente, problemas de autenticação, exposição de segredos, entradas não confiáveis e casos de borda. A revisão deve diferenciar bug comprovável de hipótese de risco.

12.3 Testes antes da entrega

Como o MVP não executa código arbitrário, a etapa de teste é uma revisão estática e uma sugestão de testes. O agente pode escrever casos unitários, testes de integração, comandos de lint e cenários manuais. A UI apresenta esse material como recomendação. Uma futura sandbox precisa pegar a saída, criar um workspace temporário, instalar apenas dependências permitidas, limitar recursos e executar os testes em ambiente isolado.

12.4 Exemplo de checklist técnico

- o código compila com a versão declarada da linguagem?
- todas as funções chamadas existem?
- os tipos de entrada e saída são coerentes?
- erros de rede, arquivo e autenticação são tratados?
- entradas do usuário são validadas?
- segredos aparecem no código ou nos logs?
- testes cobrem caminho feliz e falhas previsíveis?
- a solução depende de uma versão ou serviço não informado?
- a resposta deixou claro o que foi apenas sugerido?

13 Anexo D — Análise incremental de arquivos

O navegador não deve enviar um arquivo de centenas de megabytes inteiro para uma única chamada de modelo. A estratégia implementada é incremental: primeiro o arquivo é identificado, depois é lido em partes, cada parte recebe um resumo local e o resumo acumulado é usado como contexto para a próxima parte.

13.1 Pseudofluxo

```
arquivo selecionado
├─ ZIP? — sim → listar entradas
|
|           ├─ texto/código → chunks
|           └─ binário ———→ preservar metadados
└─ não → texto/código? → chunks ou preservação

para cada chunk:
    enviar {nome, extensão, índice, total, conteúdo, resumoAnterior}
    receber {resumo, riscos, símbolos, perguntas}
    acumular resumo

resultado:
    mapa de arquivos + visão geral + riscos + próximos passos
```

A separação entre chunkIndex, totalChunks e summary permite que o modelo saiba se está no começo, no meio ou no fim. Ela também torna possível mostrar progresso no CodeLab. O sistema não promete tamanho infinito: memória do navegador, limite HTTP, orçamento do modelo, tempo de processamento e tamanho do ZIP continuam sendo limites físicos.

13.2 Binários

Um PDF, DOCX, XLSX, imagem, áudio ou vídeo não deve ser convertido em texto por tentativa ingênua. Sem parser, o comportamento correto é preservar nome, tamanho, MIME type e bytes para exportação, informando que a análise semântica depende de um extrator. Essa distinção evita respostas inventadas a partir de bytes ilegíveis.

13.3 Segurança do upload

A versão de produção deve aplicar limites de tamanho, quantidade de arquivos, profundidade de ZIP, número de entradas e razão de expansão. Deve bloquear ZIP bombs, caminhos como `../../../../arquivo`, links simbólicos perigosos e nomes que sobrescrevam arquivos do sistema. A análise deve ocorrer em storage temporário e com expiração, nunca em um diretório compartilhado sem isolamento.

14 Anexo E — Empacotamento e exportação

A exportação é feita no navegador para manter o MVP simples e reduzir a necessidade de storage. O conteúdo atual do editor é transformado em bytes, anexos são convertidos em entradas e fflate gera um ZIP. O PDF é montado com jsPDF a partir do texto e de metadados básicos.

14.1 Estrutura do ZIP

```
maklayn-project/  
├─ README.md  
├─ maklayn-analysis.md  
├─ src/  
│   └─ current-code.txt  
└─ attachments/  
    ├─ documento-1.ext  
    └─ projeto.zip
```

O README gerado deve explicar o objetivo do pacote, a data da exportação, o arquivo principal, limitações da análise e comandos sugeridos. O relatório não deve afirmar que dependências foram instaladas ou que testes passaram apenas porque o ZIP foi criado.

14.2 Nomes e colisões

Arquivos com o mesmo nome precisam ser desambiguados. Uma política simples é preservar o nome original e acrescentar um sufixo incremental: arquivo (2) .pdf. Caminhos de ZIP devem ser normalizados para impedir traversal. A exportação deve rejeitar bytes acima do limite do navegador e avisar antes de gerar um pacote muito grande.

14.3 Evolução para storage

Em produção, a exportação deve ser feita por tarefa assíncrona quando houver muitos arquivos. O frontend enviaria referências de storage, não bytes inteiros. Uma fila criaria o pacote, gravaria o resultado com URL assinada e expirável e notificaria o usuário. O banco armazenaria nome, tamanho, hash, status, proprietário, prazo de retenção e vínculo com o projeto.

15 Anexo F — Modelo de dados e persistência

O schema inicial preserva usuários do starter e deixa a porta aberta para entidades de domínio. Uma versão completa pode adotar as tabelas abaixo.

Tabela	Campos principais	Relações
projects	id, ownerId, title, status, createdAt	um usuário possui muitos projetos
conversations	id, projectId, title, mode	uma conversa pertence a um projeto
messages	id, conversationId, role, content, createdAt	uma conversa possui muitas mensagens
documents	id, projectId, storageKey, mime, size, hash	um projeto possui anexos
sources	id, projectId, url, title, verifiedAt	fontes rastreáveis por projeto
jobs	id, projectId, kind, status, progress, error	análise e exportação assíncronas
auditEvents	id, actorId, action, target, createdAt	trilha de auditoria

Todas as queries devem filtrar pelo proprietário ou por uma regra explícita de compartilhamento. IDs públicos não substituem autorização. O servidor deve impedir que um usuário altere ownerId, leia storageKey de outro usuário ou descubra a existência de projetos por diferenças de tempo de resposta.

15.1 Migrações

Migrações devem ser aditivas sempre que possível: criar tabela, criar índice, preencher dados compatíveis e só depois remover campos antigos. Cada migração precisa de nome determinístico, revisão por pares e teste em banco vazio e banco já populado. Em deploy, duas versões do servidor podem coexistir durante a troca; por isso, mudanças incompatíveis exigem período de compatibilidade.

16 Anexo G — Modelo de segurança

A segurança da Maklayn possui quatro fronteiras: navegador, API, modelo e infraestrutura. O navegador é ambiente não confiável; qualquer valor enviado pelo cliente pode ser alterado. A API deve validar tudo novamente. O modelo é um componente probabilístico; sua saída precisa ser tratada como texto não confiável. A infraestrutura deve limitar o impacto de arquivos e tarefas demoradas.

16.1 Riscos principais

Risco	Exemplo	Mitigação
prompt injection	Documento tenta fazer o agente ignorar regras.	delimitar dados, manter instruções fora do documento, revisão
segredo exposto	Token aparece em código anexado.	redação, alerta, não persistir, revogar
ZIP malicioso	entrada com caminho traversal ou expansão extrema.	normalização, limites, sandbox
abuso de API	muitos pedidos custosos.	rate limit, quotas, filas
acesso indevido	ID de projeto de outro usuário.	autorização por query e ownership
XSS	resposta contém HTML perigoso.	Markdown sanitizado e CSP

A proteção não deve ser feita apenas no prompt. Prompt é uma camada de orientação; autenticação, autorização, validação, limites de recurso e sanitização precisam existir no código. Logs devem evitar conteúdo completo de documentos, tokens, cookies e prompts que possam conter dados pessoais.

16.2 Privacidade

O usuário precisa saber o que é armazenado, por quanto tempo e como excluir. Para LGPD, o produto deve separar dados necessários para operar de dados opcionais para melhorar a experiência. Exportação deve produzir uma cópia legível; exclusão deve remover registros, objetos e índices derivados conforme a política de retenção.

17 Anexo H — Estratégia de testes

O projeto possui testes Vitest para partes do starter e scripts de verificação de tipos e build. A evolução deve organizar testes em camadas.

17.1 Camadas

- **Unitários:** funções de chunking, normalização de caminhos, escolha de extensão, construção de prompts e cálculo de progresso.
- **Contrato:** inputs inválidos devem gerar erro previsível; respostas devem seguir o shape esperado.
- **Integração:** router tRPC com contexto autenticado, fallback do LLM e autorização por usuário.
- **Componente:** CodeLab deve adicionar arquivo, abrir ZIP, mostrar progresso e exportar.
- **End-to-end:** login, nova conversa, assistência de código, anexos e exportação em navegador.
- **Segurança:** traversal, ZIP bomb, XSS, rate limit, cookie e acesso entre usuários.
- **Build:** instalação limpa, `pnpm check`, `pnpm test`, `pnpm build` e `readiness`.

17.2 Casos de borda

O conjunto de testes deve cobrir arquivo vazio, arquivo sem extensão, nome Unicode, arquivo maior que o limite, ZIP vazio, ZIP aninhado, entrada binária, falha parcial de agente, timeout do modelo, resposta vazia, prompt sem contexto, múltiplos cliques, refresh durante exportação e ausência de credencial externa.

17.3 Critério de entrega de código

Um código pode ser considerado pronto para o usuário quando possui objetivo claro, dependências declaradas, instrução de execução, tratamento de erro, teste ou roteiro de validação e aviso explícito sobre o que não foi executado. “Funcional” deve significar que a solução é coerente e reproduzível, não que o sistema a executou automaticamente.

18 Anexo I — Build, deploy e ambientes

O desenvolvimento usa `tsx watch` para executar TypeScript e Vite para servir o frontend. O build de produção separa responsabilidades: Vite compila o cliente e esbuild empacota o entrypoint do servidor. O Dockerfile instala dependências, gera os artefatos e inicia o processo de produção respeitando PORT.

18.1 Ambientes

O ambiente local é adequado para desenvolvimento e testes rápidos. O Preview é um ambiente de validação visual e integração com o runtime WebDev. Produção precisa de banco, storage, variáveis de serviço, logs, health check e política de escala. O mesmo código não deve presumir que arquivos escritos no container sejam duráveis.

18.2 Health check

GET /api/health deve responder sem autenticação com status 2xx quando o processo está pronto para receber tráfego. O endpoint não deve expor segredos, strings de conexão ou estado interno detalhado. Um health check de liveness pode confirmar apenas que o processo está vivo; readiness pode verificar dependências críticas com timeout curto.

18.3 CI sugerida

```
checkout
-> pnpm install --frozen-lockfile
-> pnpm check
-> pnpm test --run
-> pnpm build
-> imagem/container
-> smoke test /api/health
-> publicação condicionada
```

A CI deve falhar se houver segredo versionado, lockfile inconsistente, erro de tipo, teste quebrado ou build incompleto. A publicação deve ser separada da criação do artefato para permitir revisão antes do deploy.

19 Anexo J — Observabilidade e operação

Uma plataforma de IA precisa explicar não apenas erros de software, mas também estados de modelo, fila e serviço externo. Cada chamada pode receber um `requestId`, `projectId` anonimizado e `operation`, sem registrar o conteúdo integral. Métricas úteis incluem latência, taxa de erro, tokens, tamanho de input, tamanho de output, falhas por agente, tempo de análise por chunk e exportações concluídas.

19.1 Logs estruturados

Um log operacional deve ser legível por máquina:

```
{
  "event": "ai.code_assist.completed",
  "operation": "review",
  "durationMs": 1840,
  "agents": 2,
  "partialFailure": false
}
```

Não deve conter token, cookie, conteúdo sensível ou o documento inteiro. Para depuração, o sistema pode guardar um hash e uma referência temporária, sujeita à política de retenção.

19.2 Estados de job

Análise e exportação grandes devem usar `queued`, `running`, `partial`, `completed`, `failed` e `cancelled`. O progresso precisa ser monotônico e nunca ultrapassar 100%. O usuário deve poder cancelar tarefas pendentes; o servidor deve ignorar resultados tardios de uma tarefa cancelada.

19.3 Incidentes

Um playbook mínimo deve identificar impacto, congelar publicação, verificar health check, observar taxa de erro, desabilitar feature flag problemática, preservar evidências sem dados pessoais e comunicar o status. Depois, deve registrar causa raiz, correção e ação preventiva.

20 Anexo K — Mapa de módulos da interface

A interface é organizada por intenção de uso, não por tecnologia interna.

Módulo	Uso	Elementos técnicos
Visão geral	Retomar tarefas e ver projetos.	cards, navegação, estado local
Chat	Perguntar, explicar, resumir e salvar.	mutation tRPC, streamdown, toasts
Pesquisa	Organizar fontes e hipóteses.	cards de referência, filtros, estado demonstrativo
Arena	Comparar respostas e avaliar.	dois modelos, critérios, score local
Código	Construir e revisar software.	CodeLab, editor, árvore, terminal visual
ENEM	Estudar e revisar redação.	trilhas, questões, competências
Projetos	Acompanhar espaços de trabalho.	lista, favoritos, tags e ações

A escolha de módulos permite que o mesmo backend de IA seja reaproveitado com prompts e interfaces diferentes. Pesquisa e ENEM podem evoluir para ferramentas especializadas sem duplicar autenticação, feedback ou exportação.

20.1 Acessibilidade

O MVP utiliza elementos semânticos, rótulos `aria-label`, foco visível, navegação por teclado, contraste de superfícies e responsividade. A próxima auditoria deve verificar ordem de foco, leitura por screen reader, tamanho de alvo, mensagens de erro associadas a campos e equivalência entre mouse, teclado e toque.

21 Anexo L — Glossário técnico

API: interface de programação usada para comunicação entre partes do sistema.

Backend: código executado no servidor, responsável por autenticação, regras, chamadas ao LLM e persistência.

Chunk: trecho menor de um arquivo grande usado para análise incremental.

Drizzle: ORM usado para definir schema e migrações de banco em TypeScript.

Fallback: comportamento alternativo usado quando um serviço principal falha ou não está disponível.

LLM: modelo de linguagem de grande porte usado para gerar e revisar texto ou código.

MVP: versão mínima viável, suficiente para demonstrar fluxos centrais sem afirmar que toda infraestrutura de produção está pronta.

RAG: recuperação de documentos relevantes antes da geração de uma resposta.

React: biblioteca usada para construir a interface por componentes.

tRPC: camada que permite chamar procedimentos tipados entre cliente e servidor.

Sandbox: ambiente isolado para executar código com limites e sem acesso indevido ao sistema.

ZIP traversal: ataque em que o nome de uma entrada tenta escapar da pasta de extração com `../`.

22 Encerramento técnico ampliado

A Maklayn combina uma interface de trabalho, uma API tipada e um fluxo de IA que privilegia contexto e revisão. O projeto já demonstra uma base coerente para evoluir, mas a documentação mantém uma separação honesta entre geração e execução, entre arquivo preservado e arquivo interpretado, e entre protótipo local e serviço de produção.

A principal decisão arquitetural foi deixar as fronteiras explícitas. O frontend pode oferecer uma experiência rápida; o backend pode controlar credenciais e regras; o LLM pode sugerir soluções; os agentes podem revisar uns aos outros; e uma futura infraestrutura pode executar, persistir e observar as tarefas com segurança. Essa composição permite que o produto cresça sem transformar cada nova funcionalidade em uma exceção.

O criador registrado permanece **Jhon Maklayn Mozer dos Santos**, e a data de início declarada permanece **12/09/2026**.